OGR_RESTO : 8. Rapprochement inverse (N-1 → N)

Cette étape inverse la logique des étapes précédentes. Au lieu de partir des extractions N pour chercher dans N-1, on part de la base validée N-1 et on cherche quelles nouvelles extractions N correspondent à chaque restaurant déjà connu.

8.1 — Contexte et objectif

- Différence fondamentale avec les étapes précédentes :
  - Étapes 5-6
    - Direction : N → N-1
    - Point de départ : Extractions N
    - Question : "Cette fiche N existait-elle en N-1 ?"
    - Résultat : Flag sur les fiches N (codens_n1, pas_garde_n1)
    - Utilité : Récupérer l'historique
  - Étape 8
    - Direction : N-1 → N
    - Point de départ : Base validée N-1
    - Question : "Cette fiche N-1 existe-t-elle encore dans les sources N ?"
    - Résultat : Flag sur les fiches N-1 (PJ_N, TA_N, STOCK_N, FERM_N)
    - Utilité : Détecter disparitions/confirmations

8.2 — Préparation de la base N-1

- Chargement de la base N-1 complète
- Nettoyage de l'adresse
  - Pourquoi nettoyer N-1
    - La base N-1 a peut-être été créée avec d'anciennes règles de nettoyage
    - On applique les mêmes transformations que sur les extractions N (étape 2)
    - Garantit la compatibilité entre N-1 et N pour le rapprochement
- Renommage du codens
  - Pourquoi ?
    - La table N-1 a un champ codens (identifiant permanent)
    - On le renomme codens_n1 pour éviter confusion avec le futur codens qui sera calculé à la fin
    - codens_n1 = identifiant de la campagne N-1
- Flag extraction
  - extrac = numéro d'extraction/campagne
  - 5 = code pour "fiche issue de N-1" (convention du projet)
  - Permet de différencier les fiches N-1 des fiches N dans la fusion finale
- Suppression colonnes temporaires :
  - id_pj_1, id_pj_2, id_ta_1, etc. = identifiants des sources N-2, N-3... (historique)
  - date_data = date de l'extraction N-1 (obsolète)
  - date_inser = date d'insertion N-1 (obsolète)
  - Résumé → On nettoie pour ne garder que l'essentiel
- Passe 1
  - Logique :
    - cle_complete
    - cle_complete2
    - cle_comp_ss_num
    - cle_pcompl2_ss_num
    - les extrac PJ N non rapprochées
  - Exemple concret :
    - Cas : Match → Condition : a=1 et b=1 → Action : PJ_N=1 + output BDD_RESTO_N1 → Résultat : Fiche N-1 existe dans PJ N
    - Cas : N-1 seule → Condition : a=1 et b=0 → Action : output BDD_RESTO_N1 → Résultat : Fiche N-1 pas dans PJ (pas grave)
    - Cas : PJ seule → Condition : a=0 et b=1 → Action : output pj_reste → Résultat : Nouvelle fiche PJ (pas en N-1)
    - BDD_RESTO_N1
      - Ligne 42 : "LEBISTROT", "12 RUE REPUBLIQUE", "75011 PARIS" → cle_complete = "LEBISTROT12RUEREPUBLIQUE75011PARIS" → PJ_N = . (vide au départ)
      - pj (source PJ 2025) : Ligne 101 : "LE BISTROT", "12 RUE REPUBLIQUE", "75011 PARIS" → cle_complete = "LEBISTROT12RUEREPUBLIQUE75011PARIS"
    - Après fusion :
      - BDD_RESTO_N1 ligne 42 : → PJ_N = 1 ✅ (restaurant confirmé dans PJ N)
      - pj ligne 101 : → Retirée de pj, ne va PAS dans pj_reste (car matchée)
      - Si PJ ligne 101 n'avait PAS matché :
        - BDD_RESTO_N1 ligne 42 : → PJ_N reste vide (pas confirmé par PJ)
        - pj ligne 101 : → Va dans pj_reste (nouvelle fiche à traiter)
- Passes 2-4
  - Filtre préliminaire → Ne traite QUE les fiches PJ non matchées à la passe 1 (pj_reste)
  - Nouvelle clé : cle_complete2 (au lieu de cle_complete)
  - Exemple concret :
    - BDD_RESTO_N1 ligne 42 : "RESTAURANTGAULLE", "8 RUE GENERAL DE GAULLE", "75016"
      - → cle_complete = "RESTAURANTGAULLE8RUEGENERALDEGAULLE75016PARIS"
      - → cle_complete2 = "RESTAURANTGAULLE8RUEGAULLE75016PARIS" (dernier mot)
      - → PJ_N = . (pas de match passe 1)
    - pj_reste (fiches PJ non matchées passe 1) : "RESTAURANT GAULLE", "8 RUE DE GAULLE", "75016"
      - → cle_complete = "RESTAURANTGAULLE8RUEDEGAULLE75016PARIS" ← Différent !
      - → cle_complete2 = "RESTAURANTGAULLE8RUEGAULLE75016PARIS" ← IDENTIQUE !
    - Résumé → BDD_RESTO_N1 ligne 42 : → PJ_N = 1 ✅ (match sur cle_complete2)
- Passe 3
  - Usage : détecte les fiches sans numéro de voie
  - Exemple :
    - N-1 : "PIZZERIA NAPOLI", "5 PLACE VICTOR HUGO", "75016" → cle_comp_ss_num = "PIZZERIANAPOLI PLACEVICTORHUGO75016PARIS"
    - PJ N : "PIZZERIA NAPOLI", "PLACE VICTOR HUGO", "75016" (sans n°) → cle_comp_ss_num = "PIZZERIANAPOLI PLACEVICTORHUGO75016PARIS"
    - → Match passe 3 !
- Passe 4
  - Usage : la plus large (sans n° ni complément, avec dernier mot voie)
  - Exemple :
    - N-1 : "BISTROT PARIS", "12 RUE DE LA REPUBLIQUE BAT A", "75011" → cle_pcompl2_ss_num = "BISTROTPARISRUEREPUBLIQUE75011PARIS"
    - PJ N : "BISTROT PARIS", "RUE REPUBLIQUE", "75011" (sans n° ni complément) → cle_pcompl2_ss_num = "BISTROTPARISRUEREPUBLIQUE75011PARIS"
    - → Match passe 4 !
- Création de pj_reste_fin
  - fiches PJ non matchées après les 4 passes
  - On récupère les fiches complètes de pj (avec toutes les colonnes) dont la cle_pcompl2_ss_num est dans pj_reste_4
  - Résultat → pj_reste_fin = nouvelles fiches PJ (jamais vues en N-1)

8.3 — Rapprochement avec PJ (4 passes)

8.4 — Rapprochement avec TA

- (identique à PJ)

8.5 — Rapprochement avec STOCK

- (code manuel, même logique)

8.6 — Rapprochement avec FERM

- Passe 0 spéciale : rapprochement sur SIREN !
  - Pourquoi commencer par le SIREN ?
    - SIREN = identifiant unique officiel de l'entreprise
    - Si un restaurant N-1 a le même SIREN qu'un établissement fermé dans SIRENE → fermeture avérée (très fiable) → C'est un match fort, même si l'adresse ou le nom ont changé
  - Exemple :
    - BDD_RESTO_N1 ligne 42 : "LE BISTROT", "12 RUE REPUBLIQUE", "75011 PARIS" / siren = "123456789" / FERM_N = . (vide au départ)
    - FERM (SIRENE fermés 2025) : "BISTROT PARISIEN", "10 RUE REPUBLIQUE", "75011 PARIS" (adresse changée !) / siren = "123456789" ← MÊME SIREN / datederniertraitementetablisseme = "2024-06-15" (fermé en juin 2024)
    - Après passe 0 sur SIREN : BDD_RESTO_N1 ligne 42 : → FERM_N = 1 ⚠️ (établissement fermé dans SIRENE) / Conséquence : Vérifier si établissement encore ouvert (présent dans SIRENE fermés)
- Passes 1-5 → Même pattern que STOCK, sauf qu'on ne traite que les SIREN non matchés à la passe 0.
- Colonnes ajoutées :
  - PJ_N → Restaurant présent dans PJ N → 1 ou vide
  - TA_N → Restaurant présent dans TA N → 1 ou vide
  - STOCK_N → Restaurant présent dans STOCK N (SIRENE actifs) → 1 ou vide
  - FERM_N → Restaurant présent dans FERM N (SIRENE fermés) → 1 ou vide
- Exemples de combinaisons :
  - Ex 1 : PJ_N : 1 / TA_N : 1 / STOCK_N : 1 / FERM_N : → ✅ Confirmé partout (restaurant actif, bien visible)
  - Ex 2 : PJ_N : 1 / TA_N : 1 / STOCK_N : / FERM_N : → ✅ Confirmé PJ+TA (actif mais pas dans SIRENE ou SIREN manquant)
  - Ex 3 : PJ_N : / TA_N : / STOCK_N : 1 / FERM_N : → ⚠️ STOCK seul (actif SIRENE mais peu visible en ligne)
  - Ex 4 : PJ_N : / TA_N : / STOCK_N : / FERM_N : 1 → ⚠️ FERM seul (fermé dans SIRENE, pas d'autre source) → Probable fermeture
  - Ex 5 : PJ_N : 1 / TA_N : 1 / STOCK_N : / FERM_N : 1 → ⚠️ PJ+TA mais FERM (sources contradictoires) → Vérification manuelle requise

8.7 — État de BDD_RESTO_N1 après tous les rapprochements

8.8 — Tables de "restes" (nouvelles fiches)

- Après tous les rapprochements, on a 4 tables de nouvelles fiches :
  - pj_reste_fin → Nouvelles fiches PJ (pas en N-1)
  - ta_reste_fin → Nouvelles fiches TA (pas en N-1)
  - stock_reste_fin → Nouveaux STOCK (pas en N-1)
  - ferm_reste_fin → Nouveaux FERM (pas en N-1)

8.9 — Pourquoi ce rapprochement inverse est crucial ?

1. Détection des disparitions
2. Hiérarchisation des fiches
   - Ordre de priorité (code à l'étape suivante) :
     - Fiches N-1 confirmées par plusieurs sources → couleur_fin = 1 (validation auto)
     - Fiches N-1 confirmées par une source → couleur_fin = 2 (OK rapide)
     - Fiches N-1 non confirmées + nouvelles fiches N → couleur_fin = 3-4 (validation manuelle)
3. Enrichissement progressif
4. Détection des incohérences

Presented with xmind